Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Inginerie Software și Automatică

Lucrare de laborator nr. 2

Profesor: Torică Eugeniu
Student: Zlatovcen Bogdan, grupa TI-251M

Chișinău, 2026

1 Implementarea sistemului cu RabbitMQ

În această lucrare a fost realizată decuplarea componentelor unui sistem cu ajutorul brokerului de mesaje RabbitMQ. Aplicația rezolvă același crypto-puzzle ca în laboratorul 1: se caută un nonce care, adăugat la șirul `Hello World`, produce un hash SHA-256 cu un număr prestabilit de zerouri hexazecimale la început.

Sistemul este format dintr-un producător, implementat în Ruby, și trei aplicații de tip computer. Acestea rulează în containere separate și comunică prin RabbitMQ. Două aplicații de tip computer au fost scrise în limbaje diferite față de varianta inițială: Python, ca limbaj script based, și Java, ca limbaj compilat.

Tabela 1: Componentele implementate în laborator

Fișier	**Limbaj**	**Rol în sistem**
`ruby_server.rb`	Ruby	producătorul care trimite sarcinile către workeri și primește primul răspuns valid
`ruby_computer.rb`	Ruby	worker de referință, adaptat pentru partiționarea spațiului de nonce-uri
`python_computer.py`	Python	consumer RabbitMQ care calculează SHA-256 pentru partiția primită
`Computer.java`	Java	consumer RabbitMQ compilat, cu parsare JSON minimală și client AMQP Java

Distribuirea sarcinii se face prin trimiterea unui mesaj pentru fiecare worker. Fiecare mesaj conține dificultatea, textul de bază, identificatorul workerului și numărul total de workeri. Workerul cu identificatorul `worker_id` caută doar nonce-urile din secvența `worker_id`, `worker_id + worker_count`, `worker_id + 2 * worker_count` etc. Această strategie este asemănătoare cu împărțirea spațiului de căutare în OpenMPI, unde fiecare proces primește o partiție a problemei.

Pentru oprirea workerilor care nu au găsit soluția a fost folosit un exchange fanout numit `crypto-puzzle-stop`. După ce serverul primește primul răspuns valid prin mecanismul `reply_to` și `correlation_id`, publică un mesaj de oprire. Fiecare worker verifică periodic coada de stop și se oprește dacă soluția a fost deja găsită de alt worker.

Tabela 2: Serviciile rulate prin Docker Compose

Serviciu	**Descriere**
`rabbitmq`	brokerul de mesaje, cu porturile AMQP și management expuse
`lab2_producer`	containerul din care rulează serverul Ruby
`computer_ruby`	worker Ruby
`computer_python`	worker Python
`computer_java`	worker Java, compilat și pornit în container

2 Rezultatele măsurate

Rulările au fost efectuate cu trei workeri activi: Ruby, Python și Java. Timpul este măsurat de serverul Ruby din momentul trimiterii sarcinilor până la primirea primei soluții valide.

Tabela 3: Rezultatele Lab 2 cu RabbitMQ și 3 workeri

n	**Soluție găsită**	**Timp execuție**
4	`Hello World107105`	4 204 ms
5	`Hello World1717360`	81 544 ms
6	`Hello World1439621`	73 573 ms

Pentru $n = 5$, soluția găsită în Lab 2 este diferită de soluția secvențială din Lab 1. Acest lucru este normal, deoarece există mai multe nonce-uri valide, iar ordinea de căutare se schimbă atunci când spațiul este împărțit între workeri diferiți.

Tabela 4: Comparație orientativă cu rezultatele din Lab 1

n	**Lab 1 secvențial**	**Lab 1 OpenMP, 4 fire**	**Lab 2 RabbitMQ, 3 workeri**
4	40 ms	10 ms	4 204 ms
5	236 ms	68 ms	81 544 ms
6	480 ms	139 ms	73 573 ms

Rezultatele arată că soluția cu RabbitMQ este mult mai lentă decât implementările C++ din laboratorul 1. Diferența este explicată de faptul că în Lab 2 calculul este realizat în servicii eterogene, cu overhead de broker, serializare JSON, cozi, containere și verificarea semnalului de oprire. Scopul principal al laboratorului 2 nu este obținerea vitezei maxime, ci decuplarea componentelor sistemului.

3 Probleme tehnice întâlnite

În timpul implementării au apărut mai multe probleme legate de distribuirea corectă a sarcinilor și de oprirea workerilor după găsirea soluției.

4 Comparație RabbitMQ cu OpenMPI din Lab 1

Atât Lab 2, cât și OpenMPI din Lab 1 împart spațiul de căutare între mai multe unități de execuție. Diferența principală este nivelul la care se face distribuirea. OpenMPI este orientat spre calcul paralel performant, cu procese coordonate explicit. RabbitMQ este orientat spre decuplarea serviciilor, unde producătorul și consumatorii pot fi scriși în limbaje diferite și pot rula independent.

Pentru $n = 6$, în Lab 1 OpenMPI au fost măsurați următorii timpi: 556 ms cu un proces, 379 ms cu două procese, 354 ms cu patru procese, 519 ms cu opt procese, 195 ms cu 16 procese și 188 ms cu 32 de procese. Chiar și în cea mai lentă configurație, OpenMPI rămâne mult mai rapid decât varianta RabbitMQ, deoarece folosește același algoritm C++ compilat și un model de comunicare mai apropiat de calculul paralel.

Concluzia comparației este că OpenMPI este alegerea mai bună atunci când obiectivul principal este viteza calculului paralel. RabbitMQ este mai potrivit atunci când obiectivul principal este arhitectura distribuită decuplată, integrarea mai multor limbaje și separarea clară între producători și consumatori.

Tabela 5: Probleme tehnice și soluțiile aplicate

Problemă	**Rezolvare**
Un singur consumer putea prelua toate mesajele din coadă	Serverul publică un mesaj pentru fiecare partiție, iar fiecare limbaj rulează într-un container consumer separat
Workerii fără soluție în partiția lor puteau rula la infinit	A fost adăugat exchange-ul fanout `crypto-puzzle-stop`; după primul răspuns valid, ceilalți workeri primesc semnal de oprire
RabbitMQ putea avea probleme la pornire pe Mac cu volum persistent local	În varianta simplificată brokerul pornește fără maparea volumului de date problematic
Java avea nevoie de comunicare AMQP și parsare JSON	A fost folosit clientul `amqp-client-5.20.0.jar`, iar câmpurile JSON au fost parsate minimal în `Computer.java`
Răspunsul trebuia corelat cu cererea trimisă de server	A fost păstrat modelul RPC cu `reply_to` și `correlation_id`

Tabela 6: Comparație de performanță cu OpenMPI din Lab 1

n	**Lab 1 OpenMPI, 4 procese**	**Lab 2 RabbitMQ, 3 workeri**	**Diferență aproximativă**
5	96 ms	81 544 ms	849× mai lent
6	354 ms	73 573 ms	208× mai lent

Tabela 7: Avantaje și dezavantaje RabbitMQ vs OpenMPI

Criteriu	**RabbitMQ, Lab 2**	**OpenMPI, Lab 1**
Cuplare	componente decuplate temporal și tehnologic; workeri în Ruby, Python și Java	procese coordonate strict, de obicei același program și aceeași logică de execuție
Scalare	se pot adăuga containere sau servicii consumer	se pornește programul cu `mpirun -np N` sau pe un cluster
Comunicare	mesaje asincrone, cozi și model RPC prin broker	comunicare explicită prin primitive MPI
Avantaje	flexibilitate, mix de limbaje, toleranță mai bună la servicii independente	performanță ridicată pentru calcule numerice și partiții uniforme
Dezavantaje	latență mare, serializare, configurare broker și costuri de infrastructură	mai puțină flexibilitate tehnologică și cuplare mai strânsă între procese

5 Analiza produselor Queue și Message Broker

În continuare sunt comparate câteva produse și servicii folosite pentru cozi de mesaje sau comunicare asincronă între componente.

Tabela 8: Comparație între produse Queue și Message Broker

Produs	**Restricții tehnice**	**Preț orientativ**	**Documentație și exemple**
RabbitMQ	AMQP, exchange-uri și cozi; clusterizarea este posibilă, dar necesită configurare atentă	open source; servicii managed plătite în funcție de resurse	documentație foarte bună și tutoriale oficiale pentru mai multe limbaje
Redis Pub/Sub	model fire-and-forget; fără garanții de persistență ca într-o coadă clasică	open source; servicii cloud cu nivel gratuit sau plătit	documentație bună, dar Pub/Sub nu acoperă toate cazurile de broker persistent
Amazon SQS	serviciu AWS managed, model pull, limitări de dimensiune și latență ale mesajelor	pay-per-use, potrivit pentru aplicații cloud	documentație foarte bună și SDK-uri pentru multe limbaje
Apache Kafka	log distribuit cu throughput mare; operare mai complexă decât RabbitMQ pentru proiecte mici	open source; servicii managed plătite	documentație amplă, dar curbă de învățare mai mare
NATS	broker lightweight pentru microservicii; semantică mai simplă decât Kafka	open source; opțiuni cloud managed	documentație clară, orientată spre sisteme cloud-native

Pentru acest laborator, RabbitMQ este o alegere potrivită deoarece oferă direct modelul de coadă, exchange fanout și RPC cu `reply_to`. Într-un sistem real, alegerea brokerului ar depinde de volumul de mesaje, cerințele de persistență, latență, infrastructura folosită și nivelul de operare acceptat.

6 Concluzii

Laboratorul a demonstrat modul în care un broker de mesaje poate decupla componentele unui sistem. Serverul Ruby nu depinde direct de implementarea workerilor, iar workeri diferiți pot fi scriși în Ruby, Python și Java, atât timp cât respectă același protocol de mesaje.

Comparativ cu OpenMPI din Lab 1, soluția RabbitMQ este mult mai lentă pentru crypto-puzzle, deoarece introduce overhead de comunicare, serializare și coordonare prin broker. Totuși, avantajul principal al RabbitMQ nu este performanța brută, ci flexibilitatea arhitecturală: componente independente, limbaje diferite, posibilitatea de a adăuga sau elimina workeri și separarea clară între producător și consumatori.

Prin această lucrare au fost evidențiate trei tipuri de decuplare: funcțională, deoarece serverul nu

execută direct calculul; temporală, deoarece mesajele trec prin broker; și tehnologică, deoarece aplicațiile de tip computer pot fi implementate în limbaje diferite.